

How the real-flight navigation works, block by block

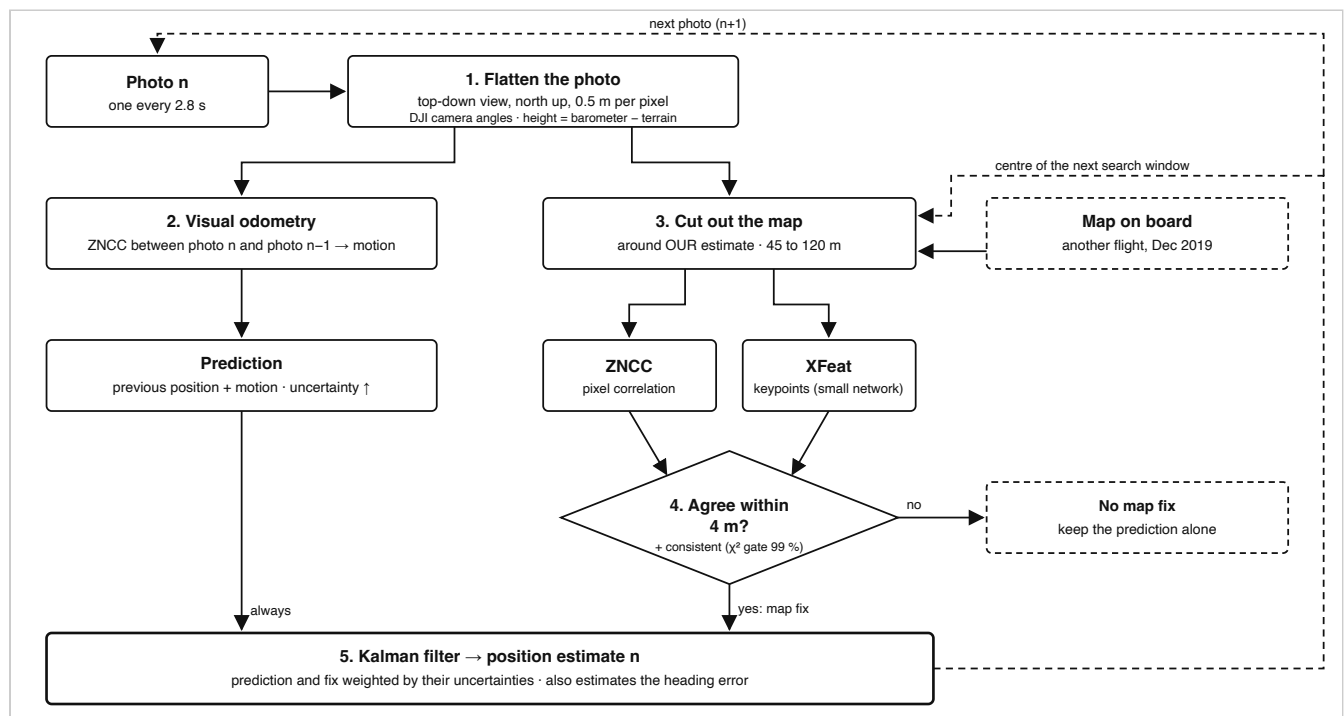
Written for the team, 4 October 2026. It explains the Tuniu real-flight result from the inside: the loop, every building block with its settings and why it was chosen, how it was tested, what that proves and what it does not, and the questions a jury is likely to ask. Every number comes from the results documents in this folder (tuniu-step1-results.md, tuniu-level2-results.md, tuniu-anti-cheat.md, tuniu-twin-results.md) and from the code in experiments/.

In one sentence. On a real drone flight in Taiwan, GPS is cut after 2 min 08 s. For every photo, our system measures how far the ground moved since the previous photo, and looks for the photo on a map made 8 months later. It keeps a map position only if two independent methods agree. Over 4 km: **3.3 m** median error, against **54 m** without the map, and **0 wrong positions accepted out of 1,646**.

Contents

1. [The loop](#)
2. [Every building block](#)
3. [How we tested it](#)
4. [What it proves, and what it does not](#)
5. [Jury questions, with answers](#)
6. [Glossary](#)

1. The loop



Every photo goes down two paths:

1. **Always** (left): measure how far the ground moved since the previous photo. This is the **prediction**. It is slightly wrong every time, and the errors add up.
2. **When possible** (right): recognise the place on the map. This is the **map fix**. It counts only if two independent methods agree.
3. The **Kalman filter** combines the two according to how reliable each one is. Its position estimate is where the map is cut out for the next photo.
4. Without any map fix, only the left path remains. That is the **red track** in the video.

2. Every building block

For each block: what it does, how (with the settings in the code), why this choice, what was ruled out, its limit, and the question it is likely to raise.

Block 0 · The inputs

Input	Real or simulated	Details
Photos	Real	DJI Phantom 4 RTK, 11 April 2019, 271 photos of 20 MP, one every 2.8 s, about 100 m above take-off (57 to 107 m above the ground: the site is a hillside), camera tilted 30° forward from vertical. Read in grey levels and decoded 8 times smaller (684 px wide), which is enough to work at 0.5 m per pixel.
Camera angles	Real, but computed by DJI with GPS on	Gimbal heading, pitch and roll, read from each photo. A small camera mounting error (−1° pitch, −0.5° heading, −1° roll) is measured on the first 46 photos and added.
Barometer	Simulated	The DJI photos do not carry the raw barometer. We take the RTK altitude and add a real barometer's noise: 0.30 m white noise, a 0.112 m/√s random walk, a slow ramp, and an unknown offset levelled before the cut. One draw per seed.
Terrain	Real	Copernicus GLO-30: one height every 30 m.
Map on board	Real, another flight	OpenAerialMap orthophoto of 12 December 2019, made from another drone flight, not a satellite (3.5 cm per pixel, CC BY 4.0), resampled to 0.5 m per pixel. Its georeference is about 2 m off (+2.0 m east, −0.3 m north); that offset is measured on the first 46 photos.

Likely question: "Where would the map come from in a real mission? A satellite?" For Taiwan, no satellite is needed: the national surveying agency (NLSC) publishes an aerial orthophoto of the whole island, about 0.27 m per pixel, as an open tile service, with its website data under the Open Government Data License (OGDL-Taiwan 1.0). Whether bulk offline use of those tiles is covered still has to be confirmed. Free satellite images are too coarse (Sentinel-2: 10 m per pixel, our whole error budget). Our test map is a drone orthophoto, 8 months after the flight, used at 0.5 m per pixel. The national orthophoto is finer, but older and taken from a plane in other light, so it may be harder. Not tested yet.

Block 1 · Flatten the photo

What it does. The camera looks forward and down; the map is seen from straight above. The photo is turned into a top-down, north-up view at the map's scale (0.5 m per pixel) so that the two can be compared.

How. For every cell of a 0.5 m ground grid, compute where it appears in the photo and read the pixel there. The computation uses the DJI camera model (lens distortion included), the camera angles, and the

height above ground (barometer minus the terrain under the current estimate). The ground is not flat: it follows the Copernicus terrain. Only the ground up to 100 m ahead of the drone is kept; further away, distortions grow fast.

What each input brings (one photo at a time, seed 0, ZNCC at 1 m per pixel):

Change	Photos recognised
No camera angles (photo not flattened)	5 %, and all 12 positions are wrong (about 53 m)
Heading 3° off / 7° off	27 % / 19 %
No barometer (100 m assumed everywhere)	22 %
Barometer minus terrain (normal)	28 %
Perfect height (upper bound)	29 %
Flat ground / ground following the terrain	19 % / 34 %

Takeaways. The camera angles are essential. The terrain matters a lot: over the whole flight, with flat ground, the system loses the map in 20 runs out of 20. The barometer helps and is almost as good as a perfect height.

Likely question: "How do we know the barometer isn't fake?" It is simulated, and we say so everywhere: the DJI photos only carry an altitude already fused with GPS, not the raw barometer. What is not made up is its error: the noise model is fitted on a real barometer log (45 min, Pixhawk, Zurich Urban MAV dataset) and reproduces its measured drift. And the result does not hinge on it:

- photo by photo, with no barometer at all, recognition drops from 28 % to 22 %; with a perfect height it is 29 %;
- over the whole flight (exploratory stress test, 5 seeds), a barometer 3 times noisier than the real one gives 4.5 m instead of 3.2 m, and one seed loses the map once; 5 times noisier gives 9.0 m and 3 seeds of 5 lose it. Details: [tuniu-level2-results.md](#).

Block 2 · Visual odometry (the motion)

What it does. Measures how far the drone moved between two photos, about 20 m.

How. Two consecutive flattened photos overlap by about 80 %. We look for the shift that lays the second one on the first, by correlation (ZNCC, block 4), to a tenth of a pixel. The far 30 % of the second photo, which the first one did not see, is left out.

Checks. The measurement is refused if the correlation peak is below 0.5 (before the cut, every bad pair scored at most 0.28 and every good pair at least 0.72), or if the speed exceeds 16 m/s, the Phantom 4 RTK's maximum. The drone is then assumed to keep its last speed along the camera heading: the fallback.

Numbers. The measurement is accepted for 82 % of photo pairs: 89 % on straight legs, 41 % in turns. Noise measured before the cut: 0.87 m per photo (set to match the drift observed over 20 photos). The fallback is off by about 1.1 m on straight legs and 7.7 m in turns, 10 to 20 m over a whole turn.

Why this choice. The dataset has no inertial unit (accelerometers and gyroscopes), so the camera is the only motion sensor. ZNCC is fast and needs no training.

Limit. Turns: the camera rotates and the photos barely overlap. The Level 2 report names this the first cause of error: 205 of the 267 photos with an error above 25 m are in a turn or just after one.

Likely question: "Why no IMU?" The published flight has no IMU log. On a real drone the gyroscopes would measure the turns, exactly where the camera fails. That is Alessandro's ESKF, the next step.

Block 3 · Cut out the map (the search window)

What it does. The map is not searched everywhere, but in a square centred on **our own estimate**, never on the truth.

Size. The half-width is the larger of 45 m and 3 times the filter's uncertainty, capped at 120 m.

- **45 m:** the size of the photo-by-photo test (start drawn within ± 40 m, plus 5 m of margin).
- **3 times the uncertainty:** if the filter is honest, the true point is inside about 99 % of the time. When the filter is less sure, the window grows.
- **120 m at most:** the larger the window, the more look-alike places, so the higher the risk of a wrong fix.

"**Losing the map**" means the true point leaves the window. It happened in none of the 20 seeds. At the worst moment, 3.5 m of margin was left.

Block 4 · ZNCC on the map (method 1)

What it does. Slides the flattened photo over the map window and scores, at every position, how well the patterns match. The best position is the map fix.

How. ZNCC means zero-mean normalised cross-correlation (OpenCV `matchTemplate`). "Normalised": the mean brightness is removed and the result is divided by the contrast, so an April sun against a December sun does not matter. It tries 5 rotations (-4° to $+4^\circ$) and 3 scales (-6% , 0 , $+6\%$) to absorb small angle and height errors.

Why this choice. A classical method, explainable end to end, with no training and no licence issue. 0.14 s per photo on one laptop core.

Limit. ZNCC **always** returns an answer, even where the place cannot be recognised (forest). It needs a check: block 6.

Block 5 · XFeat on the map (method 2)

What it does. XFeat is a small neural network (Potje et al., CVPR 2024, Apache-2.0). It finds distinctive points (corners, blobs) in the photo and in the map, and describes them so they can be paired.

How.

1. Up to 2,048 points per image, paired between the photo and the map.
2. The transformation that maps the photo onto the map is fitted (a homography, with a robust rejection of bad pairs, MAGSAC).
3. The result is refused if that transformation is implausible: scale outside 0.8 to 1.25, rotation above 12° , stretching above 1.3, or fewer than 8 pairs.

The model is used as is, without retraining, on the CPU: 0.13 s per photo.

Why XFeat. It is designed for small embedded processors, and it works on a completely different principle from ZNCC: isolated points instead of whole patterns.

Ruled out.

- **XFeat alone:** 36 % of photos recognised, but 65 wrong positions, up to 314 m off. Not reliable on its own.
- **ALIKED + LightGlue:** 66 % recognised, but 4 wrong (one seed only).
- **RoMa:** the best in published benchmarks, but too heavy; its accuracy was not tested.

Block 6 · The agreement rule (the heart of the reliability)

The rule. A map fix is accepted only if ZNCC and XFeat give two positions **less than 4 m apart**. The ZNCC position is kept. No threshold was tuned.

Why it works. The two methods fail for different reasons: one compares pixel patterns, the other isolated points. They rarely fail **at the same place**.

Why 4 m. A good map fix is typically about 2.5 m from the truth, so two good fixes land a few metres apart, while two independent mistakes almost always land far apart.

Written in advance. The rule was committed to git (commit 8e64a50, 17:04) before the XFeat results of seeds 1 to 19. Seed 0 was used to design it and is left out of the evaluation.

Result (photo by photo, 19 seeds): 30.2 % of photos recognised, **1 wrong out of 1,290**, 4 decoys accepted out of 12,825. For comparison, XFeat alone: 65 wrong.

Likely question: "Why two methods?" One method alone makes too many mistakes: XFeat alone accepts 65 wrong positions. Requiring two different methods to agree brings that down to 1 in 1,290.

Block 7 · The Kalman filter

What it keeps. Three numbers: the position (east, north) and the **heading error**, because a wrong heading turns every measured motion. It also keeps the uncertainty of each.

Predict (every photo). Position = previous position + measured motion, turned by the estimated heading error. The uncertainty grows by the odometry noise (0.87 m) or the fallback noise (up to 7.7 m in turns). The heading error may drift slowly (0.1° per $\sqrt{s}$).

Check (99 % chi-square gate, threshold 9.21). A map fix that lands too far from the prediction, given both uncertainties, is refused. Honest finding: over the whole flight this gate never stopped a wrong fix. It only turned down 19 correct fixes, after turns. The agreement rule (block 6) does the real filtering.

Correct. A weighted average of the prediction and the map fix (map-fix noise 1.78 m, measured before the cut). The uncertainty drops back. With a simulated heading drift of up to 7.3°, the filter recovers the heading to within 0.65°.

Start. The RTK position at the cut, within ± 1 m. That is the real situation: the drone knows where it was when its GPS went down.

Why Kalman. After the cut the position is known to within 1 m, so there is a single hypothesis to track, not several. A Kalman filter is the simple, fast, standard choice, and it gives the uncertainty that sizes the search window. A particle filter is only needed when the drone could be in several places at once.

Limit. It is slightly over-confident: its real error is a little larger than the one it states (consistency index 1.49, where 1 would be perfect).

Block 8 · Calibration before the cut (photos 1 to 46)

During the 2 min 08 s with GPS on, the system measures:

- the camera mounting error;
- the map offset;
- the noise of the odometry, of the fallback and of the map fixes;
- the barometer offset.

Nothing is tuned on the 225 test photos. A real drone could do the same at take-off, while it still has GPS.

Block 9 · The digital twin (a simulation checked against reality)

What it is. A 3D model of the site, built by OpenDroneMap from the 297 photos of another flight (September 2019). Inside it, a virtual camera is placed at exactly every position and angle of the April flight, and renders what it would see. Option "realistic camera": cloud shadows, haze, darker corners, blur, noise and JPEG compression.

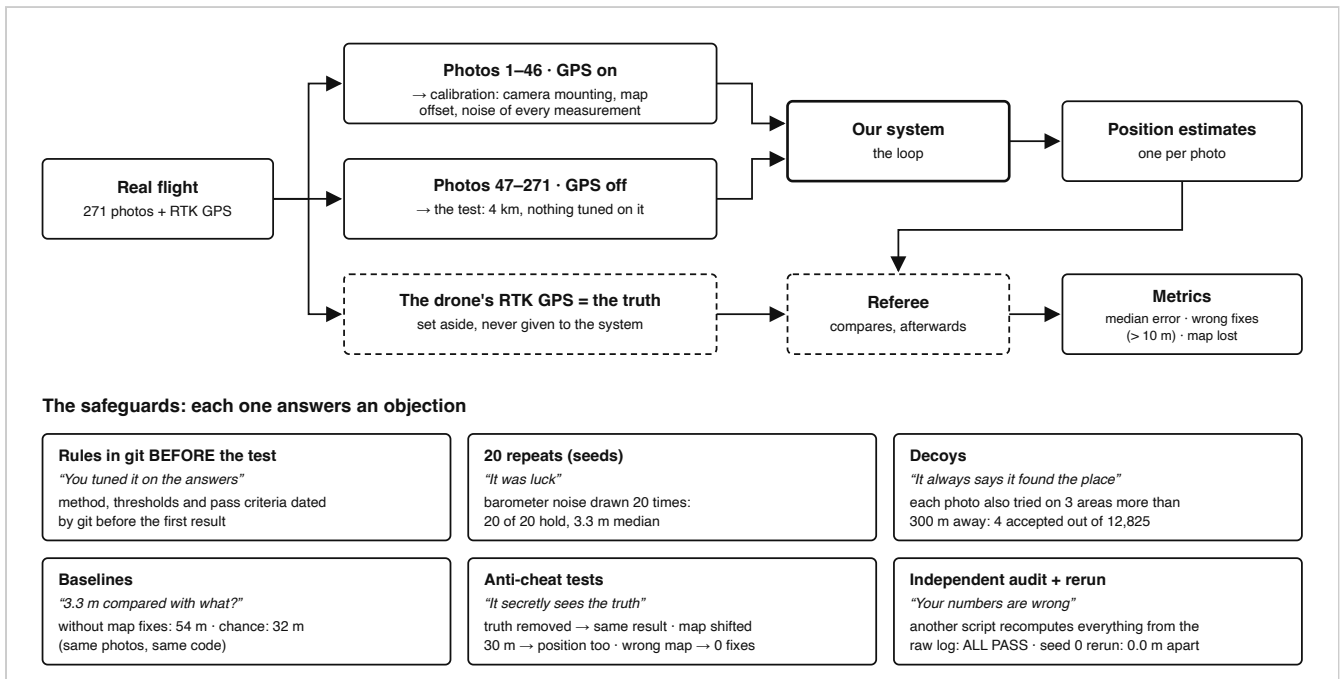
The test. Same code, same map, same settings. The results on the real photos and on the rendered images are compared on 4 criteria written before the runs (commit 38b4cd7).

Result. All 4 criteria are met: 2.81 m in simulation against 3.15 m for real, 36.7 % map fixes against 36.5 %. The simulation is slightly optimistic in the median but harsher in the worst cases: it loses the map in 3 seeds out of 5, against none for real.

What it is for. Testing what a single recorded flight cannot vary, knowing how far the simulation can be trusted. Example: with a photo every 0.5 s, the worst error halves (22 m instead of 46 m). Later, letting our estimate **steer** the drone, which a recording cannot do.

Limit. The twin was built on the 2.5D model. The full 3D model computed on 4 October has only been used for images so far.

3. How we tested it



The two test levels

	Level 1: one photo at a time	Level 2: the whole flight without GPS
Question	Can a single photo be placed on the map?	Does the system hold 4 km on its own?
Starting point	True position ± 40 m, drawn at random	Its own estimate, never the truth
Criteria written in advance	≥ 30 % of photos recognised · 0 wrong by more than 10 m · ≤ 1 % of decoys accepted	Map never lost in at least 18 seeds of 20 · median error ≤ 10 m
Date of the rules	commit 8e64a50, 17:04	commit ac18f70, 18:39; first result at 18:40
Result	30.2 % · 1 wrong out of 1,290 · 4 decoys out of 12,825	20 of 20 · 3.3 m · 0 wrong out of 1,646

The three anti-cheat tests (5 seeds each)

Test	A cheating system would...	Measured
Truth removed: GPS files deleted, GPS metadata stripped from the photos, opening the truth files blocked	crash or change its output	identical to within 0.0000000005 m
Map shifted 30 m east	stay 3 m from the truth	map fixes land 29.8 m east; median error becomes 29.2 m

Test	A cheating system would...	Measured
Wrong map (shifted 250 m, or another place)	keep a few metres of error	0 map fixes; output identical, to the bit, to the track without map fixes

Together they show that **all the gain comes from the map fixes**, and that the map fixes follow the map, not a hidden truth. Details: tuniu-anti-cheat.md.

4. What it proves, and what it does not

It proves that on this flight the result is real, honestly measured, and due to the method:

- the numbers are right (independent audit);
- the system does not see the truth (anti-cheat tests);
- the gain comes from the map (wrong map);
- nothing was tuned on the answers (dated rules, calibration on photos 1 to 46);
- it is not luck (20 seeds, decoys, comparison with chance).

It does not prove:

1. **That it works elsewhere.** One flight, one site, daylight. A second site with frozen settings is **the** missing test.
2. **That it works with real GPS-free sensors.** The barometer is simulated and the DJI heading is computed with GPS on. With a simulated heading drift, 3 wrong map fixes get through (10.5 to 16.9 m) and the margin falls to 0.3 m.
3. **That "0 wrong out of 1,646" is a strong guarantee.** It is the same photos replayed 20 times, and they overlap by 80 %. The number of truly different situations is much smaller.
4. **That the drone can steer with it.** This is a replay: the path is the one flown in 2019, and our estimate does not change it.
5. **That it runs on board.** The two map matchers take 0.27 s per photo on a laptop (with 2.8 s between photos), but nothing has been measured on an onboard computer.

5. Jury questions, with answers

How to answer in 20 seconds: a number first ("We measured it: ..."), one sentence of explanation, then the limit in our own words ("That's a limit we state ourselves: ..."). If it was not tested, say so: "We haven't tested that yet; the next step is ...".

Question	Answer to give
How do you know it isn't faked?	We built three tests a cheater would fail. We removed the truth files: same result. We shifted the map 30 metres east: our position moved 30 metres east. We gave it the wrong map: zero map fixes. So the position comes from the map, not from hidden GPS.
Why a 2019 flight and not your own data?	We had no drone of our own, so we used a real flight published by its pilot, with centimetre-level RTK GPS. Anyone can download it and check our numbers.
3.3 metres compared with what?	Compared with the same system with map fixes switched off: 54 metres, and 163 metres at the end of the flight. A random guess gives 32 metres.
Your barometer is simulated. How do we know it isn't fake?	It is simulated, and we label it everywhere, because the DJI photos don't store the raw barometer. Its noise is fitted on a real barometer log. We also stress-tested it: with a barometer three times worse we get 4.5 metres instead of 3.2; it only breaks at five times worse.
The heading comes from DJI, with GPS on. Isn't that cheating?	That's a limit we state ourselves, our main one. With a simulated heading drift of up to 7 degrees we stay at 3.2 metres and never lose the map, but three wrong fixes get through, the worst about 17 metres off.
Where would the map come from in a real mission? A satellite?	Our test map is a drone orthophoto from another flight, eight months later. For Taiwan, the national surveying agency publishes an aerial orthophoto of the whole island at about 0.27 metres per pixel, under an open licence. It is finer than what we use, but older, and we haven't tested it yet.
What happens over forest?	Trees all look alike, so there are gaps of up to 81 seconds without a map fix. The error grows to 20 or 40 metres, then the system locks back on when it sees roads or roofs. It never accepted a wrong fix there.
What about turns?	Turns are our worst case: 14.5 metres median in turns against 2.7 on straight legs, because two photos barely overlap when the camera rotates. Gyroscopes are the obvious fix; we haven't tested that yet.
Does it run in real time, on board?	We measured 0.25 seconds per photo for the whole loop on one laptop CPU core, with a photo every 2.8 seconds. We haven't measured it on an onboard computer like a Jetson yet.
Why two matching methods?	One alone makes too many mistakes: the neural matcher alone accepted 65 wrong positions. The two methods fail for different reasons, so requiring them to agree within 4 metres brought it down to 1 in 1,290.
Why a Kalman filter, not a particle filter or end-to-end deep learning?	When GPS is lost we know where we are to within a metre, so there is a single hypothesis to track. A Kalman filter is simple, fast and explainable, and it gives the uncertainty that sizes the search on the map. The only learned part is a small off-the-shelf matcher, used as is.
Zero wrong fixes out of 1,646. Really?	Yes, but to be fair it is the same photos replayed twenty times, and they overlap, so the real guarantee is weaker than the number suggests. It needs more flights.

Question	Answer to give
Does the drone actually steer with your position?	No. This is a replay: our system says where the drone is; it doesn't fly it. Closing that loop is the next step, in the digital twin, because a recording can't react.
Would it work anywhere else?	We don't know yet: one flight, one site, daylight. The next test is a second site with every setting frozen.
What is your simulation for, if you have real data?	We replayed the same flight in a 3D model of the site, with the same code, and it matched reality on four criteria fixed in advance: 2.8 metres against 3.2. So we know how far to trust it for what one recorded flight can't vary: more photos per second, another altitude, and soon steering.
How does this compare with the 18 metres of your simulator?	They are not comparable: different data, different flights, different camera. They are two separate demonstrations.

6. Glossary

Term	Meaning
RTK	GPS corrected to centimetre accuracy. Here it is the truth, used only to score the system.
Orthophoto	An aerial image corrected to look like a map seen from straight above. It can come from a satellite, a plane or a drone; ours comes from a drone.
Flatten (rectify)	Turn a photo taken at an angle into a top-down view at the map's scale.
Visual odometry	Measuring motion by comparing two consecutive images.
Map fix	Finding the absolute position by recognising the place on the map.
ZNCC	Normalised pixel correlation: how much two images look alike, whatever the brightness.
XFeat	A small neural network that finds and pairs distinctive points between two images.
Kalman filter	The standard method that combines a prediction and a measurement according to their uncertainties.
Chi-square gate	A test that refuses a measurement too far from the prediction, given the uncertainties.
Seed	A different draw of the randomness (here, the barometer noise), with the same photos.
Decoy	A map area where the photo cannot be; the only right answer is "not found".
Pre-registration	Writing the method and the pass criteria before seeing the results, with a date proven by git.
Digital twin	The same flight replayed inside a 3D model of the site, with the same code.